

RADAR B3 PRO IA — Especificação Técnica v2

Projeto: TESTE_BRAPI (Next.js + TypeScript) **Base:** commit [03a68fa](#) **Data:** 31/08/2026
Destinatário: desenvolvimento **Status:** aprovado para implementação

0. Sumário executivo

Esta especificação cobre quatro entregas encadeadas:

PR	Entrega	Motivo
PR1	Correção do crivo fundamentalista	Regra de reprovação atual gera falso negativo em massa (PETR4 reprovada com score 65/100); Dívida Líquida/EBITDA nunca é calculada
PR2	Nova fonte de dados de opções	O endpoint atual não retorna preço nem volatilidade implícita — todo prêmio exibido hoje é teórico, e o Iron Condor é matematicamente inalcançável
PR3	Rastreador como lista de execução (3 listas)	Substituir 6 categorias de veredito por 3 listas acionáveis
PR4	Plano de trade com entrada, stop e alvo	Hoje o R:R é tautológico (sempre 1,5) e não filtra nada

A ordem é obrigatória. PR3 depende de PR1: sem a correção do crivo, empresas saudáveis falsamente reprovadas passam a aparecer na lista de VENDA.

1. Princípio arquitetural

O sistema hoje mistura três decisões independentes num único campo **verdict**, o que faz o número de estados crescer a cada nova condição. A v2 separa em três eixos:

Eixo	Responsável	Saída	Onde aparece
Classificação	análise técnica apenas	ALTA BAIXA LATERAL	Rastreador (3 listas)
Autorização	fundamentos apenas	APROVADO REPROVADO	filtro invisível, nunca exibido como categoria
Execução	volatilidade, barreiras, DTE, liquidez	estrutura com strikes, prêmios e alvos	Veredito individual

Regra: os fundamentos **não** influenciam a classificação de tendência. A tendência é um fato do gráfico. Os fundamentos decidem apenas se aquela linha entra ou não na lista.

2. Regras de negócio definitivas

2.1 Camada 1 — Crivo Fundamentalista (CNPI-P)

Pesos por categoria, total 100 pontos:

Categoria	Peso	Métricas	Pontos
Rentabilidade	35	Lucro Líquido 12M (5) + ROE (15) + Margem Líquida (15)	35
Solvência	35	Dívida Líquida/EBITDA (20) + Liquidez Corrente (15)	35
Valuation	30	P/L (15) + P/VP (15)	30

Faixas de pontuação:

Métrica	BOM (pontos cheios)	NEUTRO (metade)	RUIM (zero)
Lucro Líquido 12M	> 0	—	≤ 0
ROE	$\geq 10\%$	5% a 10%	$< 5\%$
Margem Líquida	$\geq 8\%$	0% a 8%	$\leq 0\%$
DL/EBITDA	$\leq 2,5x$	2,5x a 3,0x	$> 3,0x$
Liquidez Corrente	$\geq 1,2x$	1,0x a 1,2x	$< 1,0x$
P/L	3x a 25x	$> 25x$	$< 3x$ ou negativo
P/VP	0,5x a 4,0x	$> 4,0x$	$< 0,5x$

Política de dado ausente: métrica sem dado pontua **zero** e não gera flag. Mantida como está hoje (correta).

Regra de aprovação — ALTERAR:

APROVADO $\Leftrightarrow$ score ≥ 45

E nenhuma flag ELIMINATÓRIA

E pelo menos 1 métrica de Rentabilidade com dado real

E pelo menos 1 métrica de Solvência com dado real

Flags eliminatórias — lista fechada, exaustiva:

1. **LUCRO_NEGATIVO** — Lucro Líquido 12M ≤ 0
2. **MARGEM_NEGATIVA** — Margem Líquida $\leq 0\%$
3. **SUPERENDIVIDAMENTO** — DL/EBITDA $> 3,5x$

Qualquer outra métrica classificada como RUIM (ROE baixo, liquidez corrente $< 1,0$, P/L fora da faixa, P/VP fora da faixa) **reduz o score, mas não reprova por si só**.

Isto é a correção do defeito D1. Hoje o código usa `isApproved = score  $\geq$  45 && flags.length === 0`, e o array `flags` recebe qualquer métrica RUIM.

Resultado medido com dados reais da BRAPI: PETR4 (ROE 27,8%, margem 24,4%, score 65) é REPROVADA porque a liquidez corrente é 0,854. Empresas de capital intensivo — petróleo, elétricas, saneamento, varejo de giro rápido — operam estruturalmente com liquidez corrente < 1,0. O crivo atual elimina o setor inteiro.

2.2 Camada 2 — Classificação Técnica (CNPI-T)

Sem alteração de lógica. Alinhamento estrito, já corrigido no commit [03a68fa](#):

ALTA ⇔ preço > MM20 > MM50 > MM200

BAIXA ⇔ preço < MM20 < MM50 < MM200

LATERAL ⇔ qualquer outra combinação

Sem MM200 disponível (histórico < 200 pregões), usar MM20 e MM50 apenas. RSI(14) e MACD permanecem **informativos** — não alteram a classificação.

2.3 Camada 3 — Execução

Insumo	Origem	Uso
Call Wall / Put Wall	maior Open Interest por strike (endpoint positions)	resistência/suporte institucional, alerta ≤ 3% do spot
IV por série	endpoint analytics (impliedVolatility)	regime de volatilidade, escolha crédito vs. débito
Preço da opção	endpoint analytics (optionPrice)	prêmio real das pernas
DTE	dias úteis B3 até a 3ª sexta-feira	janela 12–35 DU para Iron Condor

Regimes de volatilidade (já em [config/rules.ts](#), sem alteração): squeeze < 16%, alta ≥ 28% ou IV/HV ≥ 1,05.

2.4 Matriz de decisão — tabela verdade completa

Esta é a regra central da v2. São 6 combinações, e todas precisam de tratamento explícito no código:

#	Tendência	Fundamentos	Lista do Rastreador	Operação autorizada
1	ALTA	APROVADO	● ALTA — Compra	Trava de alta ou compra direta, com entrada/stop/alvo
2	ALTA	REPROVADO	<i>não aparece</i>	Nenhuma — value trap
3	LATERAL	APROVADO	● LATERAL — Iron Condor	Iron Condor #20, se IV e DTE permitirem
4	LATERAL	REPROVADO	<i>não aparece</i>	Nenhuma — venda de volatilidade não autorizada
5	BAIXA	REPROVADO	● BAIXA — Venda	Trava de baixa ou venda à vista com alerta de aluguel
6	BAIXA	APROVADO	<i>não aparece</i>	Saída/stop de posição existente — exibir apenas na consulta individual

Pontos críticos de implementação:

- O filtro fundamentalista tem **polaridade invertida** na lista BAIXA: ALTA e LATERAL exigem **APROVADO**; BAIXA exige **REPROVADO**.
- A palavra "REPROVADO" **nunca** aparece na UI do rastreador. Não existe card "Bloqueado", nem contador de bloqueados, nem badge de status. O filtro é invisível.
- O caso 6 **não some do sistema**: na consulta individual de um ativo aprovado em tendência de baixa, o veredito deve exibir **SAIDA_STOP** com o texto "Tendência de baixa — saída ou stop de posição existente. Nenhuma estrutura nova autorizada."

- Casos 2 e 4 não geram card nem mensagem no rastreador. Na consulta individual, exibir `SEM_OPERACAO` com o motivo.
-

3. Alterações por arquivo

PR1 — Crivo Fundamentalista

3.1.1 `src/lib/config/rules.ts`

Ajustar os pesos e adicionar o limite eliminatório de alavancagem:

FUNDAMENTALS: {

APPROVAL_SCORE_THRESHOLD: 45,

WEIGHTS: {

PROFITABILITY: 35, // Lucro Líquido (5) + ROE (15) + Margem Líquida (15)

SOLVENCY: 35, // DL/EBITDA (20) + Liquidez Corrente (15)

VALUATION: 30, // P/L (15) + P/VP (15)

},

THRESHOLDS: {

NET_INCOME_MIN: 0, // NOVO — Lucro Líquido 12M deve ser > 0

ROE_HEALTHY: 10.0,

ROE_MIN: 5.0,

NET_MARGIN_HEALTHY: 8.0,

NET_MARGIN_MIN: 0.0,

DEBT_TO_EBITDA_HEALTHY: 2.5,

DEBT_TO_EBITDA_MAX: 3.0,

```

DEBT_TO_EBITDA_ELIMINATORY: 3.5, // NOVO — acima disto reprova

CURRENT_RATIO_HEALTHY: 1.2,

CURRENT_RATIO_MIN: 1.0,

PE_MIN: 3.0,

PE_MAX: 25.0,

P_VP_MIN: 0.5,

P_VP_MAX: 4.0,

},

},

```

3.1.2 `src/lib/services/brapi.ts` — método `getFundamentals`

Problema atual (linha 287): `debtToEbitda: item?.debtToEbitda ??`

`fin?.debtToEbitda ?? null`. Nenhum dos dois campos existe no payload da BRAPI — o valor é sempre `null` e 20 pontos do score ficam permanentemente inatingíveis (teto de 80).

Verificado no payload real de `/api/v2/stocks/financial-data` (PETR4, 30/08/2026):

```

{ "totalCash": 53764000000, "totalDebt": 676283000000, "ebitda": 273323000000,

  "currentRatio": 0.8544373, "returnOnEquity": 0.2781079, "profitMargins": 0.2438609 }

```

Verificado no payload real de `/api/v2/stocks/statistics` (PETR4):

```

{ "netIncomeToCommon": 133376000000, "trailingPE": 4.662618,

  "priceToBook": 1.1670742, "bookValue": 37.315536, "enterpriseToEbitda": 4.3312244 }

```

Alterar para:

```
// Dívida Líquida / EBITDA — calcular, não ler
```

```
const totalDebt = fin?.totalDebt ?? null;
```

```

const totalCash = fin?.totalCash ?? null;

const ebitda = fin?.ebitda ?? null;

const debtToEbitda =

    totalDebt !== null && totalCash !== null && ebitda !== null && ebitda > 0

    ? Number(((totalDebt - totalCash) / ebitda).toFixed(2))

    : null;

// Lucro Líquido 12M — campo novo no RawFundamentalData

```

```
const netIncome = stats?.netIncomeToCommon ?? null;
```

Valores de referência para teste: PETR4 → $(676283 - 53764) / 273323 = 2,28x$ (BOM). MGLU3 → $(10570 - 1762) / 3116 = 2,83x$ (NEUTRO).

Se $ebitda \leq 0$ (empresa com EBITDA negativo): retornar `null` e registrar a flag **SUPERENDIVIDAMENTO** — não dividir por número negativo, o resultado seria um múltiplo negativo interpretado como saudável.

Adicionar à interface `RawFundamentalData`:

```

netIncome?: number | null;    // Lucro Líquido 12M em R$

totalDebt?: number | null;

totalCash?: number | null;

ebitda?: number | null;

```

3.1.3 `src/lib/domain/fundamentals.ts`

1. **Adicionar bloco de Lucro Líquido** (5 pts), antes do ROE. Se `netIncome <= 0` → 0 pts + flag **LUCRO_NEGATIVO**.
2. **Reajustar pontos**: ROE 18 → 15; Margem 17 → 15; DL/EBITDA 20 (mantém); Liquidez 15 (mantém); P/L 15 (mantém); P/VP 15 (mantém). Total 100.
3. **Separar flags de eliminatoryFlags**:

```
const flags: string[] = [];    // observações, não reprovam
```



```
const eliminatoryFlags: EliminaryFlag[] = []; // reprovam
```

```
type EliminaryFlag = 'LUCRO_NEGATIVO' | 'MARGEM_NEGATIVA' |  
'SUPERENDIVIDAMENTO';
```

4. Substituir a regra de aprovação:

// ANTES

```
const isApproved = score >= approvalThreshold && flags.length === 0 &&  
availableMetricsCount >= 2;
```

// DEPOIS

```
const hasProfitabilityData = roeVal !== null || netMarginVal !== null || netIncomeVal !== null;
```

```
const hasSolvencyData = debtToEbitdaVal !== null || currentRatioVal !== null;
```

```
const isApproved =
```

```
score >= CNPI_RULES.FUNDAMENTALS.APPROVAL_SCORE_THRESHOLD &&
```

```
eliminatoryFlags.length === 0 &&
```

```
hasProfitabilityData &&
```

```
hasSolvencyData;
```

A exigência de dado real em Rentabilidade e Solvência elimina o defeito D3: hoje é possível aprovar com 47 pontos vindos só de P/L e P/VP dentro da faixa, sem nenhum dado de resultado ou de dívida — que é a definição de armadilha de valor.

5. Expor `eliminatoryFlags` no `FundamentalAnalysisResult` para a UI poder explicar o motivo na consulta individual.

PR2 — Fonte de dados de opções

3.2.1 Diagnóstico

O endpoint `/api/v2/options/positions` retorna **exclusivamente** dados de posição em aberto. Payload real (PETR4, série PETRI464, 31/08/2026):

symbol, side, strike, optionStyle, expirationDate, openInterest, openInterestChange,

coveredQuantity, blockedQuantity, uncoveredQuantity, totalPositionQuantity,

borrowerQuantity, lenderQuantity, isin, segment, distributionId, reportDate

Não existe `lastPrice`. Não existe `iv`. Consequências no código atual:

- `pos.lastPrice` é sempre `undefined` → `calculateImpliedVolatility` nunca é chamado → `ivAtm` fica `undefined` → `volRegime` é `null` → veredito `LATERAL_AGUARDAR` para **100%** dos ativos laterais. É a causa exata do "0 Iron Condor / 24 Lateral (Aguardar)" observado em produção.
- `lastPrice: pos.lastPrice || greeks.theoreticalPrice` → todos os prêmios exibidos (grade straddle, crédito líquido, breakeven, bilhete de ordem) são **preços teóricos Black-Scholes calculados com volatilidade histórica**, não preços de mercado.

3.2.2 Endpoint correto

Usar `/api/v2/options/analytics` (mesmos parâmetros de `positions`: `underlying`, `expirationDate`, `side`, `minStrike`, `maxStrike`, `date`, `limit`). Payload real por série:

```
{  
  
  "symbol": "PETRI464", "side": "call", "strike": 44.17, "optionStyle": "american",  
  
  "model": "cox-ross-rubinstein", "priceSource": "close",  
  
  "underlyingPrice": 43.55, "optionPrice": 1.07,  
  
  "impliedVolatility": 0.28729522,  
  
  "delta": 0.4794953, "gamma": 0.17611259, "theta": -13.108564, "vega": 4.0729957, "rho":  
  1.1363474,  
  
  "riskFreeRate": 0.14, "dividendYield": 0,  
  
  "confidence": "high", "nullReason": null,  
  
  "openInterest": 5592200, "openInterestDate": "2026-08-28"  
}
```

3.2.3 Alterações

src/lib/services/brapi.ts — novo método:

```
async getOptionAnalytics(  
  
  symbol: string,  
  
  expirationDate: string  
  
): Promise<OptionAnalyticsItem[]>
```

Cache com o mesmo TTL usado em **getOptionPositions**.

src/lib/domain/options-barriers.ts — **analyzeOptionPositions** passa a receber os dois conjuntos e fazer merge por **symbol**:

```
export function analyzeOptionPositions(  
  
  underlyingSymbol: string,  
  
  underlyingPrice: number,  
  
  positions: OptionPositionItem[],    // fonte de Open Interest → walls, Max Pain  
  
  analytics: OptionAnalyticsItem[],   // fonte de preço, IV e gregas  
  
  expirationDate: string,  
  
  allExpirations: B3ExpirationInfo[],  
  
  historicalPrices: number[]  
  
): OptionAnalysisResult
```

Regras do merge:

- Chave: **symbol**. Série presente em **positions** e ausente em **analytics** = série sem negociação — mantém OI (serve para wall) e fica **sem preço e sem IV** (**optionPrice: null, iv: null**). **Não preencher com preço teórico**.
- **delta, gamma, theta, vega** passam a vir de **analytics**. **Remover** o cálculo local de gregas para séries com analytics disponível.

- `calculateImpliedVolatility` (solver local) deixa de ser usado no fluxo principal. Manter a função e seus testes como fallback documentado, mas não chamá-la quando houver `impliedVolatility` da fonte.
- `impliedVolatility` vem em **decimal** (0,287). Converter para percentual (28,7) na fronteira do serviço, uma única vez.

IV ATM — substituir a heurística atual:

// Elegibilidade da série para compor a IV ATM

```
const isEligible = (o) =>
```

```
  o.confidence === 'high' &&
```

```
  o.optionPrice !== null && o.optionPrice >= 0.10 &&
```

```
  o.openInterest >= 1000 &&
```

```
  Math.abs(o.strike - spot) / spot <= 0.05; // ±5% do spot
```

// IV ATM = mediana das IVs elegíveis (call e put juntas)

// Se nenhuma série elegível → `ivAtm = null` → regime de vol = null → sem Iron Condor

Divergência call/put — se $|IV_{call} - IV_{put}|$ do strike mais próximo do spot for > 5 pontos percentuais, marcar `ivQuality: 'DIVERGENTE'` e tratar como dado não confiável (mesmo efeito de `ivAtm = null`). Indica preço desatualizado em uma das pontas.

3.2.4 Constantes hardcoded a remover

Local	Valor atual	Substituir por
<code>black-scholes.ts</code> , <code>options-barriers.ts</code>	<code>r = 0.1075</code>	<code>riskFreeRate</code> da série (payload traz 0,14)
<code>black-scholes.ts</code>	<code>q = 0.03</code>	<code>dividendYield</code> da série
<code>options-barriers.ts</code>	modelo Black-Scholes europeu para todas as séries	respeitar <code>optionStyle</code> ; quando a fonte fornecer gregas, usá-las diretamente

A B3 tem calls **americanas** (a própria BRAPI usa Cox-Ross-Rubinstein nelas). O código atual precifica tudo como europeia, o que distorce preço teórico e delta.

PR3 — Rastreador como lista de execução

3.3.1 Novo motor de decisão

Criar `src/lib/domain/operation-matrix.ts`. Este arquivo passa a ser a **única** fonte da decisão operacional.

```
export type OperationType =

  | 'COMPRA'      // ALTA + APROVADO

  | 'VENDA'       // BAIXA + REPROVADO

  | 'IRON_CONDOR' // LATERAL + APROVADO

  | 'SAIDA_STOP'  // BAIXA + APROVADO → só na consulta individual

  | 'SEM_OPERACAO'; // ALTA/LATERAL + REPROVADO → só na consulta individual

export interface OperationDecision {

  operation: OperationType;

  listedInTracker: boolean; // true apenas para COMPRA, VENDA, IRON_CONDOR

  trend: TrendType;

  fundamentalStatus: 'APROVADO' | 'REPROVADO';

  reason: string;

}

export function resolveOperation(

  trend: TrendType,

  fundamentalStatus: 'APROVADO' | 'REPROVADO'

): OperationDecision
```

Implementação — tabela verdade literal, sem lógica implícita:

trend	fundamentalStatus	operation	listedInTracker
ALTA	APROVADO	COMPRA	true
ALTA	REPROVADO	SEM_OPERACAO	false
LATERAL	APROVADO	IRON_CONDOR	true
LATERAL	REPROVADO	SEM_OPERACAO	false
BAIXA	REPROVADO	VENDA	true
BAIXA	APROVADO	SAIDA_STOP	false

3.3.2 Depreciar `generateConsolidatedVerdict`

Os 7 valores de `ConsolidatedVerdictType` saem do fluxo principal. Substituir por `OperationDecision` + os alertas como campos separados:

```
export interface AssetDecisionResult {  
  
  symbol: string;  
  
  shortName?: string;  
  
  currentPrice: number;  
  
  changePercent: number;  
  
  trend: TrendType;           // eixo 1  
  
  movingAverages: MovingAverages;  
  
  fundamentals: FundamentalAnalysisResult; // eixo 2  
  
  operation: OperationDecision; // eixo 1 × eixo 2  
  
  tradePlan?: TradePlan;      // PR4  
  
  optionStructure?: ElectedOptionStrategy; // eixo 3
```

```

barrierAlert?: OptionBarrierAlert;    // alerta, não decisão

rentalAlert?: RentalAlert;            // apenas em VENDA

updatedAt: string;

}

```

O alerta de Call Wall a $\leq 3\%$ **deixa de mudar o veredito** e passa a ser um campo de alerta exibido junto ao card. Hoje ele gera um tipo de veredito próprio (COMPRA_COM_ALERTA_BARREIRA), o que multiplica estados sem mudar a ação.

3.3.3 src/app/api/trends/route.ts

Resposta nova:

```

{

  "totalAnalyzed": 60,

  "lists": {

    "alta":  [ /* AssetDecisionResult[] — operation.operation === 'COMPRA' */ ],

    "baixa": [ /* operation.operation === 'VENDA' */ ],

    "lateral": [ /* operation.operation === 'IRON_CONDOR' */ ]

  },

  "updatedAt": "..."

}

```

Ativos com `listedInTracker: false` **não** entram na resposta. Não devolver contagem de excluídos, não devolver os ativos excluídos.

Invariante obrigatória (teste automatizado): `lists.alta.length + lists.baixa.length + lists.lateral.length <= totalAnalyzed`, e nenhum símbolo pode aparecer em duas listas.

3.3.4 `src/components/screener/ScreenableView.tsx`

- Substituir os 6 cards por 3 seções: **ALTA — Compra**, **BAIXA — Venda**, **LATERAL — Iron Condor**.
 - Remover: card "Bloqueado (Fundam.)", card "Todos os Ativos", filtro por veredito, badge de status fundamentalista, coluna de score.
 - **Estado vazio explícito** por seção: "Nenhuma operação autorizada nesta direção hoje." Não preencher com o "melhor disponível" — a lista vazia é informação válida.
 - Cada linha exibe: ticker, nome curto, preço, variação %, e o resumo da operação (estrutura ou entrada/stop/alvo).
-

PR4 — Plano de trade

3.4.1 Problema atual

`calculateRiskReward` em `indicators.ts` deriva o alvo a partir do risco:

```
const target1 = Number((spot + risk * 1.5).toFixed(2));
```

```
const rRatio = Number((((target1 - spot) / risk) || 1.8).toFixed(1));
```

O R:R resultante é sempre 1,5 por construção. **Não é uma medida, é uma constante** — não serve como filtro.

3.4.2 Método correto

O alvo vem da **estrutura de preço**, o R:R é medido depois e serve de corte:

ALTA (compra)

entrada = spot (ou rompimento da resistência imediata, se a estratégia for de rompimento)

stop = suporte imediato - $0,5 \times \text{ATR}(14)$

alvo 1 = resistência imediata

alvo 2 = segunda resistência

risco = entrada - stop

ganho = alvo1 - entrada

R:R = ganho / risco

BAIXA (venda)

entrada = spot

stop = resistência imediata + $0,5 \times \text{ATR}(14)$

alvo 1 = suporte imediato

alvo 2 = segundo suporte

R:R = $(\text{entrada} - \text{alvo1}) / (\text{stop} - \text{entrada})$

Corte: $R:R < 1,5 \rightarrow$ a operação **não é listada** no rastreador. Na consulta individual, exibir com a marcação "Relação risco/retorno insuficiente (X:1) — operação não recomendada".

```
export interface TradePlan {  
  
  entry: number;  
  
  stop: number;  
  
  target1: number;  
  
  target2: number;  
  
  riskPerShare: number;  
  
  rewardPerShare: number;  
  
  riskRewardRatio: number; // medido, não fixado  
  
  atr: number;  
  
  isViable: boolean;      // riskRewardRatio >= 1.5  
  
  method: string;        // texto explicando de onde vieram os níveis  
  
}
```

Se `calculateSupportResistance` não devolver nível válido acima/abaixo do spot, `isViable = false` e a operação sai da lista — nunca substituir por percentual arbitrário (o código atual usa `spot * 0.96 / spot * 1.05` no caso lateral).

3.4.3 Alerta de aluguel (lista BAIXA)

A BRAPI não fornece dado de aluguel. Verificado em `/api/v2/stocks/statistics` (PETR4): `sharesShort`, `shortRatio`, `shortPercentOfFloat`, `dateShortInterest` — todos `null`. Os campos `borrowerQuantity` / `lenderQuantity` do endpoint de posições referem-se à **série de opção**, não ao empréstimo da ação, e não devem ser usados como proxy.

Implementação aprovada — alerta estático, sem número:

```
export interface RentalAlert {  
  
  required: true;  
  
  message: 'Venda à vista exige aluguel (BTC). Confirmar disponibilidade de doador e taxa na corretora antes de executar. Alternativa sem aluguel: trava de baixa com opções.';  
  
}
```

Exibir sempre que a operação **VENDA** oferecer a alternativa de venda à vista. **Não estimar** taxa ou disponibilidade a partir de `floatShares` ou volume — número estimado errado é pior que ausência de número, porque o usuário dimensiona a posição em cima dele.

Ordem de preferência na lista BAIXA: trava de baixa com opções é a estrutura padrão; venda à vista é alternativa secundária, sempre acompanhada do alerta.

4. Testes obrigatórios

Todos em Vitest, no padrão dos arquivos existentes em `src/lib/domain/*.test.ts`.

4.1 Fundamentos

#	Caso	Entrada	Esperado
F1	PETR4 dados reais	ROE 0,2781 / margem 0,2439 / DL-EBITDA 2,28 / liq	APROVADO (hoje reprova)

#	Caso	Entrada	Esperado
		0,854 / P/L 4,66 / P/VP 1,167 / LL 133,4bi	
F2	MGLU3 dados reais	ROE 0,0079 / margem 0,0023 / DL-EBITDA 2,83 / liq 1,099	REPROVADO por score
F3	Prejuízo	LL = -500mm, resto excelente	REPROVADO, flag LUCRO_NEGATIVO
F4	Superendividada	DL/EBITDA 4,0, resto excelente	REPROVADO, flag SUPERENDIVIDAMEN TO
F5	EBITDA negativo	ebitda = -100mm	debtToEbitda = null + flag SUPERENDIVIDAMEN TO
F6	Só valuation (D3)	P/L 10 + P/VP 1,0, sem rentabilidade nem solvência	REPROVADO (hoje aprova com 47)
F7	Valor-limite	score exatamente 44 / 45 / 46	44 → REPROVADO, 45 → APROVADO, 46 → APROVADO
F8	Sem dado nenhum	{ }	REPROVADO, score 0
F9	Liquidez 0,99 com score alto	liq 0,99, resto excelente	APROVADO (RUIM não é eliminatório)

4.2 Matriz de operação

Teste exaustivo das 6 combinações da tabela da seção 2.4 — cada uma verificando `operation` e `listedInTracker`.

4.3 Opções

#	Caso	Esperado
O1	Série com OI e sem analytics	entra como wall, <code>optionPrice = null, iv = null</code> , sem preço teórico
O2	<code>confidence: 'low'</code>	não elegível para IV ATM
O3	<code>optionPrice < 0,10</code>	não elegível
O4	Strike a 8% do spot	não elegível (fora de $\pm 5\%$)
O5	Nenhuma série elegível	<code>ivAtm = null</code> → regime <code>null</code> → sem Iron Condor
O6	IV call 32% vs put 40% no ATM	<code>ivQuality: 'DIVERGENTE'</code> → tratado como sem IV
O7	Merge por symbol	gregas vêm de analytics, OI vem de positions
O8	DTE 11 / 12 / 35 / 36 DU	11 e 36 bloqueiam Iron Condor; 12 e 35 permitem

4.4 Plano de trade

#	Caso	Esperado
T1	R:R medido = 2,3	<code>isViable: true</code> , aparece na lista
T2	R:R medido = 1,2	<code>isViable: false</code> , não aparece na lista
T3	Sem resistência acima do spot	<code>isViable: false</code> , sem fallback percentual
T4	R:R nunca constante	dois ativos diferentes devem produzir R:R diferentes

4.5 Contrato de dado externo

Fixar os payloads reais citados nesta especificação como fixtures em `src/lib/services/__fixtures__/` e testar o parsing contra eles. Este teste é o que teria evitado o defeito D2 (campo `debtToEbitda` que nunca existiu na fonte).

Adicionar teste de campo ausente: se a BRAPI parar de enviar `ebitda`, o sistema deve degradar para `debtToEbitda = null`, e **não** quebrar nem aprovar por omissão.

5. Critérios de aceite

1. PETR4 aparece na lista correspondente à sua tendência — não é mais eliminada pelo crivo.
 2. `debtToEbitda` tem valor numérico para ativos com EBITDA positivo; score máximo alcançável é 100.
 3. Existe pelo menos um ativo classificado como Iron Condor quando há IV real disponível na série mensal mais líquida.
 4. Nenhum prêmio exibido na UI é preço teórico — todo valor monetário de opção vem de `optionPrice` da fonte, ou o campo aparece como "sem negociação".
 5. O rastreador não exibe em nenhum lugar a palavra "reprovado", "bloqueado" ou contagem de excluídos.
 6. Nenhum símbolo aparece em mais de uma lista.
 7. Consulta individual de ativo aprovado em tendência de baixa exibe `SAIDA_STOP` com a mensagem definida.
 8. Suíte completa verde, incluindo os testes das seções 4.1 a 4.5.
-

6. Decisões registradas (não são omissões)

Decisão	Justificativa
LATERAL + REPROVADO não opera	Iron Condor perde ~3× o que ganha; empresa deteriorada tem cauda gorda (follow-on, covenant, RJ), grade de strikes grosseira em papel barato, e 4 pernas ilíquidas impedem o fechamento antecipado que é a gestão da estrutura. O bloqueio é proxy para controles que o sistema não tem (limite de size, calendário de eventos, delta

Decisão	Justificativa
	hedge). Revisar quando esses controles existirem.
BAIXA + APROVADO fora do rastreador	Não há operação nova a executar; a ação correta é gestão de posição, e o sistema ainda não conhece a carteira do usuário. Preservado na consulta individual.
Alerta de aluguel sem número	A fonte não tem o dado. Integração com o arquivo diário de empréstimo de ativos da B3 fica para escopo futuro.
Solver local de IV mantido, mas fora do fluxo	Passa a ser fallback documentado. Instável em opção de centavos (1 centavo move a IV em 1,5 a 3 pontos) por usar tolerância absoluta de R\$ 0,05.
RSI e MACD permanecem informativos	Não entram na classificação de tendência, conforme especificação original.

7. Ordem de execução

PR1 (fundamentos) —→ PR3 (3 listas)

| ↑

└──→ PR2 (opções) ───┐ ───→ PR4 (plano de trade)

PR1 antes de PR3 é obrigatório. Sem a correção do crivo, empresas saudáveis falsamente reprovadas — PETR4 entre elas — passam a ser listadas como candidatas a **venda**. Hoje o defeito apenas esconde bons ativos; depois da mudança da tela, ele vira recomendação ativa de operar contra uma empresa sólida.

Sugestão de validação entre PR1 e PR3: rodar a varredura completa e comparar a distribuição de aprovados antes e depois. Se o número de aprovados não subir de forma relevante em relação aos 35 de 60 atuais, há outro problema no crivo que precisa ser investigado antes de seguir.